

04 · Synchronisation und Ressourcenverwaltung

Race Conditions, Mutexe, Deadlocks und Priority Inversion · EZS · DHBW Stuttgart

Prof. Dr.-Ing. Daniel Klünder

Folie

1 / 59

Willkommen auf der dunklen Seite der Echtzeit!

- Vierter Termin: geteilte Ressourcen auf **einem** FreeRTOS-Kern (CM4)
- Präemption ist nützlich und erzeugt konkurrierende Zugriffe
- Heute: Race, Semaphor, Mutex, Deadlock, Prioritätsinversion
- Ein Mutex dieses Kernels schützt nicht den CM7, DMA oder fremde ISRs

Notizen

1 / 59

Plattform wie VL3, soweit das Projekt sie bestätigt: NUCLEO-H755ZI-Q, Kernel auf dem CM4. Keine .ioc im Repo — UART, Takt und IRQ-Prioritäten am Cube-Projekt ablesen. Mehrkern nur als Ausblick.

Folie

2 / 59

Agenda für heute

1. Lost Update und was „atomar“ hier heißt
2. Spin, IRQ-Maske, Blockieren
3. Semaphor und Mutex, inklusive ISR-Regeln
4. Deadlock und Prioritätsinversion mit Zahlen
5. Pathfinder als belegte Fallstudie, dann Labor

Notizen

2 / 59

Dieselben Tasksets nicht mit neuen Zahlen vermischen. Das Inversionsbeispiel gilt für beide Diagramme.

Recap: Die Macht der Präemption

- Wechsel nicht nur am SysTick: auch ein Ereignis oder eine IRQ kann einen Thread bereit machen
- Im üblichen CM4F-Port führt **PendSV** den Kontextwechsel aus
- Auf einem Kern rechnen nicht zwei Threads gleichzeitig
- „Gleichzeitig“ heißt hier: verschachtelt, mit eigenen Registern

Notizen

3 / 59

Anschluss an VL2/VL3. Hohe Priorität läuft, wenn sie Ready ist. Vor ihrem Release ist sie nicht Ready.

Folie

4 / 59

Motivation: Der Kampf um den UART

- Zwei Threads wollen je eine ganze Zeile ausgeben
- `printf` schreibt nicht universell Byte für Byte in ein Datenregister
- Abhängig von Retargeting, Puffer, Bibliothekslock, HAL und Transfermodus
- Möglich: vermischte Zeichen, `HAL_BUSY`, verlorene Ausgabe — nicht „immer Buchstabensalat“

Notizen

4 / 59

Das Motivationsbild ist eine angenommene byteweise Ausgabe. Den echten Pfad im Labor prüfen.

Folie

5 / 59

Das visuelle Chaos

Angenommene byteweise Retarget-Funktion, ohne gemeinsamen Schutz:

Schritt	Temp	Hum	Leitung
1	T=21		T=21
2		H=40 \n	T=21 H=40
3	Rest C \n		Zeile zerrissen

Geschützt: formatieren im privaten Puffer, dann Acquire, **ganze** Nachricht, Abschluss, Release.

Bytes und logische Nachricht trennen. Der Mutex repariert nicht automatisch die C-Bibliothek. Alle Writer müssen denselben Schutz benutzen.

Das Problem geteilter Ressourcen

- Fehler ist schon ein verlorenes Update, kein nötiger HardFault
- `volatile` macht Read-Modify-Write nicht atomar
- Ein Mutex schließt die **beteiligten Threads** aus, nicht jede IRQ
- Lesen gehört dazu, wenn mehrere Felder zusammen einen Snapshot bilden

Frage: Hilft `volatile` gegen das verlorene Update? Nein. Es verhindert Wegoptimieren, nicht das Verschränken zweier RMW-Folgen. Data Race im Sprachmodell und der Maschinenablauf sind verwandt, aber nicht dasselbe.

Lernziele des heutigen Tages

1. Ein verlorenes Update an Registern und RAM erklären
2. Spin, Maskierung und Blockieren nach dem Konkurrenten wählen
3. Semaphor-Token und Mutex-Ownership unterscheiden
4. Dieselbe Inversion ohne und mit PIP nachrechnen
5. Pathfinder als Vereinfachung einer belegten Ursache einordnen

PIP verkürzt die Extra-Verzögerung durch Medium. Es beendet weder Deadlocks noch Starvation.

Was ist eine Race Condition?

- Das Ergebnis hängt von der Verschränkung nicht synchronisierter Zugriffe ab
- Auf einem Kern genügt eine Präemption zwischen Lesen und Schreiben
- Betroffene: Threads untereinander, oder Thread und ISR, oder ein zweiter Busmaster
- Der Mechanismus muss zu **diesen** Konkurrenten passen

Erste Frage der Matrix: Wer greift noch zu?

Das simpelste Code-Beispiel

```
static volatile uint32_t counter; /* sichtbar, aber nicht atomar  
void bump(void) { counter++; } /* kann ein Update verlieren */
```

- Vereinfachtes Compilerbild, keine Garantie für genau drei Befehle
- Ein einzelnes atomares Load oder Store ist noch kein atomares ++

Das Disassembly des Labor-Builds in die Notiz schreiben, sobald der Build existiert. Heute gibt es kein Firmwareprojekt im Repo.

Der Blick unter die Haube (Assembler)

Lehrbild, nicht das universelle Listing:

1. LDR: RAM nach Register
2. ADD: nur das Register
3. STR: Register nach RAM

Zwischen den Befehlen kann der Kern wechseln. Das Register von A bleibt in **As** Kontext, nicht in Bs.

PendSV sichert As Register. B startet mit eigenen Registern und liest den RAM erneut.

Visualisierung der Datenkorruption

Start counter = 5. Zwei Inkremente, erwartet 7.

Zeit	aktiv	Register A	Register B	RAM
1	A liest	5	—	5
2	A addiert	6	—	5
3	Wechsel, B liest	6 (gesichert)	5	5
4	B schreibt	6	6	6
5	A schreibt	6	—	6

Geschützt, gleiche Starts: A liest und schreibt fertig (RAM 6), danach B schreibt 7.

Synthetisches Maschinenmodell. Die 6 von A wandert nicht in Bs Register. `volatile` ändert diese Tabelle nicht.

Atomarität (Die Lösung des Problems)

- „Atomar gegenüber den relevanten Zugriffen“ heißt: kein sichtbarer Zwischenstand **für diese Konkurrenten**
- Das ist nicht „keine Unterbrechung überhaupt“
- Unter einem Mutex dürfen fremde Threads und IRQs weiterlaufen
- Eine kurze IRQ-Maske ist ein anderes Werkzeug mit anderem Schutzradius

Gegenseitiger Ausschluss und Interruptfreiheit nicht gleichsetzen.

Definition der Begriffe

Begriff	hier
Kritischer Abschnitt	Zugriffe, die zusammenpassen müssen
Mutex	Ausschluss von Threads, mit Owner
Semaphor	Tokenzahl, ohne Owner
Spin	aktiv pollen, bleibt Running
Blocked	keine eigenen Threadbefehle; der Kern kostet weiter Zeit

Entscheidung, kurz:

Konkurrent	Mittel
Thread/Thread, Ressource	Mutex
ISR signalisiert Thread	Semaphor oder Thread-Flag
ein Zähler, geprüft	Atomic oder sehr kurze Sektion
mehrere gleiche Plätze	Counting Semaphore
Nachricht mit Daten	Queue (nächster Termin)

Notizen

13 / 59

Ein Mutex wirkt nur, wenn jeder beteiligte Threadzugriff dieselbe Vereinbarung einhält. Atomics: Typ, Alignment und erzeugter Code prüfen; keine Peripherieregister „nebenbei“ atomar machen.

Folie

14 / 59

Schutzidee 1: Der Spinlock (Polling)

Absichtlich falsch:

```
while (lock) { }
lock = 1; /* Check-then-Set ist selbst eine Race */
```

- Ein korrektes Test-and-Set beseitigt diese Race
- Es beseitigt nicht das Fortschrittsproblem auf einem präemptiven Kern
- Für den Laborfall (H wartet auf L) ist Spin die falsche Wahl
- Nicht jedes SMP- oder Bare-Metal-Szenario ist damit verboten

Notizen

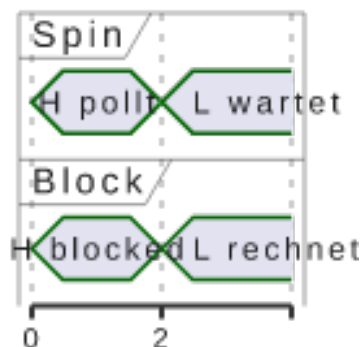
14 / 59

Zwei Ursachen getrennt halten: nicht-atomares Flag und „der Besitzer bekommt keine CPU“.

Warum Spinlocks im RTOS crashen

L hält den Lock. H wird bereit und pollt.

- Spin: H bleibt Running, L ist nur Ready und gibt nicht frei
- IRQs können trotzdem laufen
- `osThreadYield` in H gibt L **nicht** die CPU, solange H höher und Ready bleibt
- Blockieren: H wird nach erfolglosem Acquire Blocked, L kann weiterrechnen
- Das löst noch nicht die spätere Störung durch Medium



Notizen

15 / 59

„Blocked = keine eigenen Befehle“ ist richtig. „CPU-Last 0 %“ ist falsch, sobald Idle, Tick oder eine andere Aufgabe rechnet. Frage: Hilft Yield im hohen Spin? Nein.

Folie

16 / 59

Schutzidee 2: Interrupts sperren

- Eine kurze, budgetierte Sperre kann zu harten Fristen passen
- Gezählt wird die Dauer des Abschnitts plus Ein- und Austritt, nicht „ein Takt“
- Lehrwerte: 2 µs bei Frist 100 µs kann passen; 5 ms bei derselben Frist nicht
- Im ungünstigen Fall kommt die restliche Maskierungszeit zur übrigen Latenz hinzu
- `__disable_irq/__enable_irq` (PRIMASK) nicht mit den FreeRTOS-Makros gleichsetzen
- Der CM4F-Port maskiert über **BASEPRI** bis zu einer Schwelle und hebt den vorigen Stand wieder auf

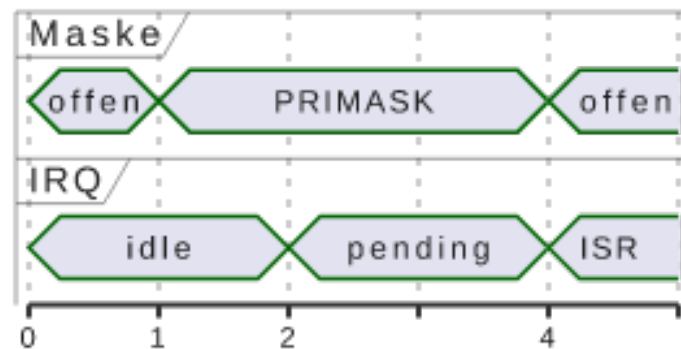
Notizen

16 / 59

`__enable_irq` löscht eine frühere Maske nicht sauber, wenn jemand anderes PRIMASK gesetzt hat. NMI, HardFault und nicht maskierte höhere IRQs fallen nicht unter denselben Schutz. Kleinere NVIC-Zahl

= höhere IRQ-Dringlichkeit, nicht die Taskkala.

Die Grenzen der Interrupt-Sperre



- Pending heißt: die IRQ wartet, DMA muss damit nicht stehen
- In der maskierten Sektion kein Delay, kein `printf`, kein blockierendes API
- Task- und ISR-Makros des Ports nicht vertauschen

Synthetische 2 μ s und 5 ms, keine Boardmessung. Andere Latenzanteile bleiben außerhalb dieses Balkens.

Die „richtige“ Lösung: OS-Blocking

- Für Thread gegen Thread: blockieren statt polieren
- Der Besitzer muss laufen dürfen, sonst gibt er nicht frei
- PIP kommt erst, wenn ein höherer Thread **am selben Mutex** wartet
- Vorher hat L nur seine eigene Priorität

OS-Blocking ist die passende Idee für das Labor, kein Universalschloss.

Zusammenfassung Block 1

- Verlorenes Update: Endwert 6 statt 7
- Spin kann den Besitzer verdrängen
- IRQ-Sperre ist ein Latenzbudget, kein Tabu und kein Freifahrtschein
- Mutex: beteiligte Threads, nicht der ganze Chip

Notizen

19 / 59

Nächster Block: Token und Ownership.

Was ist ein Semaphore?

- Zähler von 0 bis `max_count`
- `Acquire` verringert, `Release` erhöht
- Kein gespeicherter Owner: ein anderer Thread darf `Release` rufen
- ISR: `osSemaphoreRelease` ist vorgesehen; `Acquire` nur mit Timeout 0
- Zusätzlich muss die NVIC-Priorität die Syscall-Schwelle einhalten
- „ISR-safe“ heißt nicht „aus jeder Priorität“

Notizen

20 / 59

CMSIS-Spezifikation und FreeRTOS-Adapter können abweichen. Die Datei `cmsis_os2.c` im Projekt ist maßgeblich. Version hier nicht erfunden.

Typ 1: Counting Semaphore (Zähl-Semaphor)

Pool mit `osSemaphoreNew(3, 3, NULL)`:

Aktion	count
Start	3
A nimmt	2
B nimmt	1
C nimmt	0
D mit <code>timeout = 0</code>	0, Fehler, kein Zugriff
B gibt frei	1, ein Wartender kann nehmen

- Es entsteht kein zweiter freier Platz
- Der Zähler ist Verfügbarkeit, nicht die Identität eines Platzes
- Höchste Priorität unter den Wartenden, soweit das Backend so wählt; keine Fairness-garantie
- Release bei vollem Zähler kann `osErrorResource` liefern und erzeugt kein weiteres Token

Parkhaus bleibt Bild. Die Bilanz ist die Aussage. „95 % binär“ ohne Quelle streichen.

Typ 2: Binary Semaphore (Binärer Semaphor)

- `max_count = 1`
- Ressource anfangs da: `osSemaphoreNew(1, 1, NULL)`
- Ereignis, noch nichts passiert: `osSemaphoreNew(1, 0, NULL)`
- Zwei Releases vor dem nächsten Acquire: der zweite scheitert am vollen Zähler
- Zwei Hardwareereignisse werden damit nicht automatisch zweimal bearbeitet

Tokenbedeutung festlegen: „Arbeit da“, „ein Ereignis“ oder „Puffer frei“. Sonst Queue oder größerer Zähler, jeweils mit Überlaufregel.

CMSIS-API für Semaphoren

```
osSemaphoreId_t sem =
    osSemaphoreNew(1, 0, NULL); /* Ereignis, anfangs leer */
if (sem == NULL) { Error_Handler(); }

osStatus_t st = osSemaphoreAcquire(sem, 10);
if (st != osOK) {
    /* Timeout oder Parameter: Ressource nicht anfassen */
}
```

- timeout = 0: sofortiger Status, kein Blockieren
- endliche Tickzahl: Fehlerpfad Pflicht
- osWaitForever: kann dauern; das ist eine Entwurfsentscheidung

Notizen

23 / 59

Status nicht ignorieren. Ein binärer Semaphor ist kein Mutex mit Owner.

Folie

24 / 59

Was exakt passiert bei Acquire / Release?

count	Acquire	Release
> 0	zählt herunter, osOK	zählt hoch, bis max
0, Warten	Thread Blocked	weckt einen Wartenden oder zählt auf 1
= max	—	osErrorResource, Zähler bleibt

Notizen

24 / 59

Nicht jedes Acquire bei 0 blockiert. Timeout 0 kehrt sofort zurück.

Der Hauptanwendungsfall: Signalisierung

```
/* ISR, Priorität erlaubt, Handle gültig, Kernel läuft */
if (osSemaphoreRelease(sem) != osOK) {
    /* voll: Ereignis geht in diesem Tokenmodell verloren */
}

/* Worker */
if (osSemaphoreAcquire(sem, osWaitForever) == osOK) {
    bearbeite_ein_token();
}
```

- Kein `printf`, kein Mutex, kein blockierendes Warten in der ISR
- Der Worker läuft erst, wenn die ISR fertig ist und seine Priorität ihn wählen lässt
- Die CPU muss vor dem Ereignis nicht Idle gewesen sein

Notizen

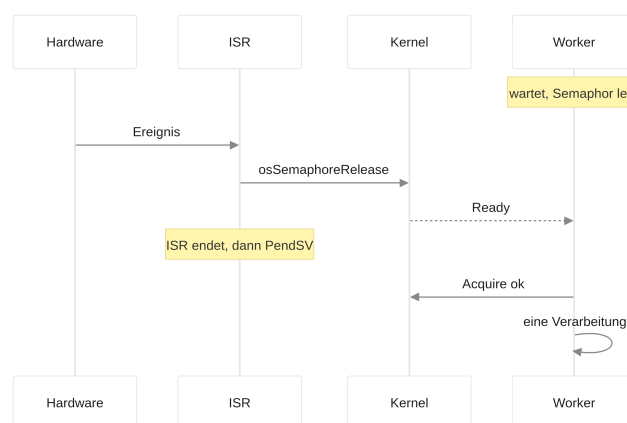
25 / 59

Thread-Flags können dasselbe Bit ebenfalls zusammenfassen. Beides ist keine verlustfreie Ereigniswarteschlange.

Folie

26 / 59

Sequenzdiagramm: Signalisierung aus der ISR



Notizen

26 / 59

Kein Thread-Kontext mitten in der noch laufenden ISR. Ein zweites Release bei max 1 kann scheitern.

Semaphoren haben kein „Ownership“

- Token merkt sich nicht, wer ihn nahm
- B darf ein Release ausführen, auch wenn A genommen hat
- Das ist für Ereignisse nützlich und für eine exklusive Ressource gefährlich
- Deshalb für den UART-Schutz ein Mutex, nicht dieses Token

Notizen

27 / 59

Binärer Semaphor mit Start 1 ist die Laborvariante **ohne** PIP, nicht „derselbe Mutex mit anderem Namen“.

Folie

28 / 59

Die Rettung: Der Mutex (Mutual Exclusion)

- Zustände: frei, oder Owner = A
- B kann nicht freigeben: die API liefert einen Fehler
- Das ist keine Hardware-Sperre gegen wilde Speicherzugriffe
- Nach erfolgreichem Acquire zugreifen, genau einmal passend freigeben
- In den gezeigten Abschnitten kein vermeidbares `osDelay`: der Mutex maskiert keine IRQs, verlängert aber die Sperrdauer

```
const osMutexAttr_t attr = { .attr_bits = osMutexPrioInherit };  
osMutexId_t m = osMutexNew(&attr);
```

Notizen

28 / 59

CMSIS-Default 0 enthält **kein** PIP-Bit. Die dokumentierte CMSIS-FreeRTOS-Anbindung erzeugt native Mutexe trotzdem mit Inheritance und ignoriert dieses Bit. Weglassen schaltet PIP dort nicht ab. Lokal `cmsis_os2.c` prüfen. `osMutexRobust` ist in dieser Anbindung nicht die Rettung.

CMSIS-API für Mutexe

```
static int use_locked(void) {
    if (osMutexAcquire(m, 5) != osOK) /* 5 Ticks = 5 ms bei 1 kHz */
        return 0; /* kein Release: wir besitzen ihn nicht */
    int ok = sensor_ok();
    if (ok) use_resource();
    osMutexRelease(m);
    return ok;
}

void Worker(void *argument) {
    (void) argument;
    for (;;) {
        (void) use_locked();
        osDelay(10);
    }
}
```

- Acquire-Fehler: Ressource nicht anfassen, nichts freigeben
- osMutexAcquire/osMutexRelease sind in CMSIS nicht aus der ISR erlaubt
- Rekursion ist ein **eigenes** Attribut osMutexRecursive
- Zweites Acquire ohne Rekursion: bei osWaitForever Self-Deadlock, bei Timeout ein Fehler

Notizen

29/59

use_locked darf zurückkehren. Worker bleibt in der Schleife. Timeout 5 Ticks sind bei 1 kHz 5 ms. Besitzdauer des Beispiels ist nur der Sensorzugriff, nicht das Delay.

Sonderfälle in der C-Programmierung

Rekursion, Owner A:

Aktion	Zähler
frei	0
A acquire	1
A acquire nochmal	2
A release	1
A release	0, jetzt darf B

- B kommt erst bei 0 dran
- Ohne `osMutexRecursive` ist die zweite Zeile kein harmloses Verschachteln

Notizen

30 / 59

Zähler, Owner und Tokenzahl nicht vermischen.

Folie

31 / 59

Best Practices für Mutexe

- Ein Mutex, eine Invariante, ein Name
- Alle konkurrierenden Threadzugriffe, auch die lesenden
- Kurzer Abschnitt; Rechnung vor dem Acquire, Warten dahinter
- Keine zweite Sperre in umgekehrter Reihenfolge
- Kein Mischen von `osMutex*` und `xSemaphore*` auf demselben Objekt

Notizen

31 / 59

Ownership verhindert die falsche Freigabe über die API. Sie verhindert nicht, dass jemand den Puffer direkt beschreibt.

Rekursive Mutexe

- Nur wenn dieselbe Aufgabe dasselbe Schloss erneut braucht
- Nicht als Ersatz für eine klare Aufrufstruktur
- Jedes Acquire braucht sein Release
- Prioritätsvererbung und Rekursion sind unabhängig wählbare Bits; der FreeRTOS-Adapter folgt den Bits nicht in jedem Punkt

Notizen

32 / 59

Im Zweifel nicht rekursiv, sondern die kritische Arbeit in eine Funktion ohne erneutes Acquire legen.

Folie

33 / 59

Zusammenfassung: Mutex vs. Binärer Semaphor

	Mutex	Binärer Semaphor
Gegenstand	Ressource mit Owner	Token oder Ereignis
Release durch anderen	API-Fehler	möglich
ISR	weder Acquire noch Release	Release ja, Acquire nur Timeout 0
PIP	gewünscht und im Adapter oft immer an	nein
Start	frei	0 oder 1, je nach Bedeutung

Notizen

33 / 59

Folie 50 der alten Fassung gilt für den genannten Adapter, nicht als CMSIS-Gesetz für jedes RTOS.

Folie

34 / 59

Die Katastrophe: Wenn Locks böse werden

- Deadlock: eine **Gruppe** wartet zyklisch
- Andere Threads und IRQs können weiterlaufen
- „System freeze“ ist eine mögliche Folge, kein Zwang
- Prioritätsinversion ist kein Deadlock: der Besitzer könnte weiterrechnen, wird aber verdrängt

Starvation aus VL3: High bleibt Ready und gibt die CPU nicht her. Inversion: High ist am Lock blockiert, Low kommt nicht zum Release.

Der Deadlock (Verklemmung)

T1 hält A und will B. T2 hält B und will A.

Pfeil	Bedeutung
Task → Ressource	wartet auf
Ressource → Task	zugeteilt an

Zyklus: T1 → B → T2 → A → T1.

Ablauf: T1 nimmt A, T2 nimmt B, T2 wartet auf A, T1 wartet auf B. Beide Locks sind gehalten, beide Threads Blocked.

Einzelinstanz-Locks. Ein Zyklus in einem anderen Ressourcenmodell ist nicht automatisch derselbe Beweis.

Die vier Coffman-Bedingungen

1. Mutual exclusion: die Ressource gehört einem
 2. Hold and wait: halten und auf die nächste warten
 3. No preemption: **niemand nimmt die Ressource zwangsweise weg** — das ist nicht „keine CPU-Präemption“
 4. Circular wait: der Zyklus oben
- Alle vier zusammen, in diesem Lockmodell.

CPU-Präemption läuft weiter und verhindert den Deadlock nicht.

Vermeidung von Deadlocks

Globale Ordnung: immer A vor B.

- T2 wartet auf A und besitzt B noch nicht
- T1 kann B nehmen, dann beide freigeben
- Rang auch für Sperren, die eine Bibliothek innen nimmt
- Timeout nur mit Rollback; allein verhindert er keine inkonsistenten Daten und keinen Livelock
- PIP öffnet keinen Zyklus

Notizen

37 / 59

Freigabe in umgekehrter Reihenfolge ist eine gute Konvention, die Ordnung beim Nehmen ist die Regel.

Folie

38 / 59

Der subtile Feind: Priority Inversion

Lehrmodell, ein Kern, Kosten der API vernachlässigt, $H > M > L$, keine weiteren Threads:

- L ab $t = 0$, nimmt den Lock, braucht 5 ms CPU bis zum Release
- H ab $t = 1$, will denselben Lock, danach 1 ms CPU; $D_H = 10$ ms, absolut $t = 11$ ms
- M ab $t = 2$, 20 ms CPU, fasst den Lock nicht an
- Vorher sind H und M nicht Ready
- Der kurze Acquire-Versuch von H zählt nicht als CPU-Zeit

Notizen

38 / 59

Dieselben Jobs in beiden Bildern. M hat denselben CPU-Bedarf, nicht dieselbe Busy-Wait-Dauer aus der Wanduhr.

Die Anatomie der Inversion (Schritt 1 und 2)

Ohne PIP, binärer Semaphor als Ressourcentoken:

- 0–2 ms: L läuft und hält; bei 1 ms blockiert H
- 2–22 ms: M läuft, L ist nur Ready
- 22–25 ms: L restliche 3 ms, Release
- 25–26 ms: H rechnet

Wartezeit von H auf den Lock: 24 ms. Antwortzeit $R_H = 25$ ms. Fertig bei 26 ms, nach der Deadline 11 ms.

Medium ist das anschauliche Extra, nicht die Bedingung jeder Inversion. „Unbegrenzt“ gilt, wenn Medium dauerhaft Ready bleibt. Bei diesem begrenzten Job ist die Zeit endlich und trotzdem zu lang.

Notizen

39 / 59

Direkte Blockierung durch L und zusätzliche Verdrängung durch M getrennt benennen. Gesamt-CPU-Arbeit: $5 + 1 + 20 = 26$ ms.

Die Anatomie der Inversion (Schritt 3 und 4)

Mit PIP, ein Mutex, nur dieser Lock:

- 0–1 ms: L mit eigener Priorität
- ab 1 ms: H blockiert, L **erbt** H
- 1–5 ms: L rechnet weiter, obwohl M bei 2 ms Ready wird
- Release bei 5 ms, H rechnet 5–6 ms
- M danach 6–26 ms

Wartezeit 4 ms, $R_H = 5$ ms, fertig bei 6 ms, vor $t = 11$ ms.

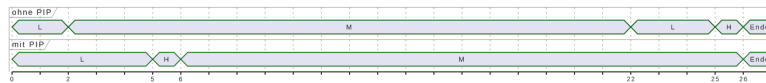
Die Erbschaft beginnt bei Hs Warten, nicht schon ab $t = 0$. PIP kürzt die Medium-Lücke. Es streicht nicht Ls restliche kritische Sektion.

Notizen

40 / 59

FreeRTOS vereinfacht die Vererbung bei mehreren gehaltenen Mutexen: L fällt nicht nach jedem einzelnen Release zwingend sofort auf die Basispriorität. Das Ein-Mutex-Bild nicht dafür ausgeben. Anbindung an VL2: Blocking gehört in die Antwortzeit.

Visualisierung der Priority Inversion



	ohne PIP	mit PIP
H wartet auf den Lock	24 ms	4 ms
R_H	25 ms	5 ms
fertig	26 ms, zu spät	6 ms, rechtzeitig

Notizen

41 / 59

Pro Kern eine laufende Aufgabe. Nach dem einmaligen Job ist Ende, keine unendliche Ready-Schleife. Deadline ist ein Lehrwert. Synthetisches Diagramm.

Folie

42 / 59

Warum ist das so katastrophal?

- H hält seine angenommene Frist nicht, obwohl H die höchste Basispriorität hat
- Die Ursache ist der Lock plus die Verdrängung von L, nicht „der Scheduler ignoriert H“
- Bei unbeschränktem Medium gibt es keine endliche Schranke mehr
- Bei bekanntem Medium bleibt die Zeit endlich und kann trotzdem die Deadline verfehlen

Notizen

42 / 59

PIP ist kein Beweis, dass jede Frist hält. Die restlichen 4 ms von L bleiben in R_H .

Folie

43 / 59

Die reale Katastrophe: Mars Pathfinder (1997)

- Didaktische Vereinfachung, keine Boardmessung und kein Patchlisting
- Quelle: Glenn E. Reeves, Bericht vom 15.12.1997
- VxWorks-„mutex semaphore“ nicht mit einem FreeRTOS-Binary-Semaphore gleichsetzen
- Keine belegte Funkstrecke und kein „Schutz war generell aus“ als ganze Geschichte

Die Fallstudie erklärt Abhängigkeit und Tracing, nicht einen neuen Mars-Vortrag.

Die Architektur des Pathfinder

- Niedrigpriorer ASI/MET-Pfad und der zeitkritische `bc_dist`-Pfad teilen den Mutex im `select/pipe-IPC`
- Mittelpriore Arbeit kann ASI/MET verdrängen, während der Mutex gehalten wird
- `bc_dist` verpasst die Frist
- `bc_sched` erkennt das und stößt den Resetpfad an

Das L/M/H-Bild ist die Vereinfachung dieser Pfade. Der Mutex lag im IPC, nicht in einer frei erfundenen Hardware.

Was auf dem Mars passierte

1. Resets und Fristfehler
2. Trace, auf der Erde nachgestellt
3. Lock-Abhängigkeit benannt
4. Getestete Änderung, wie diese Mutexe erzeugt werden
 - Ein reiner Hardware-Watchdog als einzige Erklärung ist zu grob
 - Der valide Patch änderte eine globale Option für die Erzeugung dieser Mutexe

Keine erfundenen Register und kein erfundener Quelltext des Patches.

Die Lösung: Wie rettete die NASA die Sonde?

- Priority Inheritance für die betroffenen Mutexe einschalten
- Das kürzt die zusätzliche Verzögerung, analog zum Lehrbild
- Es ist kein Freibrief für beliebige kritische Abschnitte
- Tracing war der Weg zur Ursache, nicht die Vererbung allein

Notizen

46 / 59

Dieselbe Lehre wie im Labor: ohne Nachweis, dass L den Lock wirklich hielt und H darauf wartete, ist die Geschichte keine Inversion.

Folie

47 / 59

Das Priority Inheritance Protocol (PIP)

- H wartet am Mutex, den L hält → L bekommt vorübergehend Hs Dringlichkeit
- M kann L in diesem Fenster nicht mehr verdrängen
- Nach dem Release läuft H, danach M
- Mehrere Mutexe: FreeRTOS-Vererbung vereinfacht; nicht „bei jedem Release sofort Low“ behaupten
- PIP behebt keinen Deadlock und keine allgemeine Starvation

Notizen

47 / 59

Wenn Medium nach dem Release dauerhaft Ready bleibt, verhungert L weiter. PIP ist kein Fairnessmechanismus.

Folie

48 / 59

Wie PIP das Mars-Problem löste

- Im Lehrbild: H fertig bei 6 ms statt 26 ms
- Auf dem Pathfinder: die Mutexe des betroffenen Pfads erben, `bc_dist` wird nicht mehr durch die mittelpriore Verdrängung über die Frist hinaus festgehalten — soweit der Reeves-Bericht die Ursache trägt
- Unbegrenzte mittelpriore Last bliebe ein anderes Problem

Formulierung eng am Bericht halten. Keine Totalverlust-Dramatik ohne Beleg.

Visualisierung: Priority Inheritance („Die Prio-Beule“)

Effektive Priorität von L:

- 0–1 ms: Low
- 1–5 ms: geerbt von H
- ab Release: wieder Low, H läuft

M ist ab 2 ms Ready und läuft erst ab 6 ms.

Die Beule beginnt beim Warten von H, nicht beim Start von L. Frage: Welche Verzögerung bleibt trotz PIP? Die restliche kritische Sektion von L, hier 4 ms ab Hs Versuch.

Achtung: Mutex vs. Semaphore in FreeRTOS

- Für „ohne PIP“ einen binären Semaphor mit `initial_count = 1` verwenden
- Für „mit PIP“ einen Mutex mit gesetztem `osMutexPrioInherit`
- Im genannten Adapter das Bit nicht als Abschaltknopf verstehen
- Alle New/Acquire/Release-Aufrufe und Typen wechseln, nicht nur den Handle-Namen

CMSIS-FreeRTOS Technical Data der eingesetzten Version lesen. Öffentliches `main` belegt die Laborversion nicht.

Laborvorbereitung: Chaos am UART

Zwei getrennte Versuche:

1. Nachrichtenschutz: überlappende Zeilen ohne und mit Mutex
2. Inversion: gleicher CPU-Bedarf, einmal Semaphor, einmal Mutex

Saubere Zeilen beweisen den Schutz im beobachteten Lauf, keine Deadline.

115200 Baud, 8N1, 100 Byte, synthetisch:

$$100 \times 10 / 115200 \approx 8,68 \text{ ms}$$

reine Übertragungszeit, plus Software und Warten. Der Lock kann lange gehalten sein, obwohl der C-Code kurz aussieht.

Notizen

51 / 59

IT/DMA: die Startfunktion kehrt vor dem Ende zurück. Der Puffer muss bleiben. Die ISR signalisiert; sie gibt keinen CMSIS-Mutex frei. Der Owner released im Thread.

Folie

52 / 59

Labor-Schritt 2: Die Absicherung

- Privater Puffer, dann Acquire, ganze Ausgabe, Abschluss, Release
- `osDelay` außerhalb
- Alle Writer, auch Bibliothekswege, wenn sie denselben UART benutzen
- Status von Acquire und von der Ausgabe prüfen
- Beobachtung offen lassen: Mischtext, `HAL_BUSY` oder bereits serialisierte Bibliothek

Notizen

52 / 59

Kein „immer Buchstabensalat“ als Sollprotokoll.

Labor-Schritt 3: Die Sabotage (Inversion)

- L `osPriorityNormal`, M `osPriorityAboveNormal`, H `osPriorityHigh`, sofern der Wrapper das nicht klemmt
- H und M starten blockiert an **eigenen** Gates, nicht am Test-Lock
- L nimmt den Test-Lock zuerst und meldet das
- Danach ein begrenzter CPU-Abschnitt, kein `osDelay` als Ersatz für diese Last
- M arbeitet ein begrenztes Pensum und blockiert dann
- Endlosschleife nur als eigener, beendbarer Grenzfall: nach PIP-Release kann L weiter verhungern

Notizen

53 / 59

Wenn M schon läuft, bevor L den Lock hat, ist das Starvation und keine nachgewiesene Inversion. Frage: War L wirklich Owner, und war H am Lock blockiert?

Folie

54 / 59

Labor-Schritt 4: Die Heilung

- Dieselben Freigaben und dieselbe CPU-Arbeit
- Mutex statt Semaphore, Attribute gesetzt, API vollständig getauscht
- Erwartung im Lehrsinn: H wartet nur noch auf Ls Rest, nicht auf ganz M
- Messen: L-Acquire, H-Wunsch, M-Start und -Ende, L-Release, H-Erfolg, H-Ende
- `printf` erst nach dem Lauf; GPIO, Trace oder ein geschützter RAM-Puffer
- DWT `CYCCNT` nur wenn der Zähler läuft und der Takt bekannt ist; er zählt auch fremde Zeit

Notizen

54 / 59

Das 24/4-ms-Bild nicht als Boardmessung eintragen. Build ersetzt keine Messung. Kein Projekt im Repo, deshalb hier kein übersetzter Labor-Binary.

UART, IT und DMA

- Blockierend: Timeout und Release auf jedem Pfad
- IT/DMA: Ende ist nicht die Rückkehr von HAL_UART_Transmit_IT
- Abschlusssignal in den Owner-Thread, dort Release
- Ein Logger-Task mit begrenzter Queue ist ein späteres Muster, kein Echtzeitbeweis

Notizen

55 / 59

Queue-Überlauf braucht eine Regel. Das vertieft VL5.

Folie

56 / 59

Auswertungsblatt

Feld	ohne PIP	mit PIP
L Owner vor H?		
H blockiert am Lock?		
M-CPU-Bedarf gleich?		
Wartezeit H		
R_H		
Deadline gehalten?		Lehrfrist nur im Modell
L nach Release noch verhungert?		

Notizen

56 / 59

Fragen: Warum durfte M laufen? Weil L ohne Erbschaft niedriger war. Was bleibt mit PIP? Ls restliche Sektion.

Zusammenfassung des Tages

- Lost Update: 6 statt 7; `volatile` genügt nicht
- Semaphore zählt Token; Mutex merkt sich den Owner
- Deadlock ist ein Ressourcenzyklus; Coffman-„No preemption“ meint die Ressource
- Ohne PIP: Wartezeit 24 ms, $R_H = 25$ ms; mit PIP: 4 ms und 5 ms
- Pathfinder: select/pipe-Mutex, Frist von `bc_dist`, Reset über `bc_sched`, Reeves 15.12.1997

Notizen

57 / 59

Kontrollen: Token nie negativ und nie über max. Rekursion 0-1-2-1-0. UART-Beispiel 8,68 ms. CPU-Summe beider Varianten 26 ms. Absolute Deadline 11 ms.

Folie

58 / 59

Ausblick auf Termin 5

- Queues transportieren Daten, nicht nur ein Token
- ISR schreibt nicht blockierend in die Queue
- Der Mutex von heute bleibt für den gemeinsamen Puffer zuständig, wenn die Queue ihn nicht ersetzt

Notizen

58 / 59

Mehrkern und DMA bleiben außen vor, bis das Projekt sie wirklich übt.

Folie

59 / 59

Quellen

- Arm CMSIS-RTOS2: Mutex und Semaphore
- CMSIS-FreeRTOS Technical Data der **eingesetzten** Version, nicht blind `main`
- FreeRTOS-Kernel-Book, Ressourcen und Inheritance
- `CM4F-portmacro.h`: BASEPRI, kritische Sektionen
- CMSIS-Core: PRIMASK und BASEPRI
- GCC `__atomic`, falls ein Atomic-Beispiel gebaut wird
- Glenn E. Reeves, What really happened on Mars?, 15.12.1997

Lehrdiagramme und die Zeiten $2\ \mu\text{s}$, $5\ \text{ms}$, $8,68\ \text{ms}$ sowie $24/4\ \text{ms}$ sind synthetisch.